

检查这部分的配置

用户:

Build, Execution, Deployment > Compiler

Revert changes

Resource patterns:

Use ; to separate patterns and ! to negate a pattern.

☒ Clear output directory on rebuild

Warning: if enabled, the entire contents of directories where generated sources are stored will be cleared on rebuild.

☒ Add runtime assertions for notnull-annotated methods and parameters

[Configure annotations...](#)

☒ Automatically show first error in editor

☒ Display notification on build completion

☒ Build project automatically

Only works while not running / debugging

☒ Compile independent modules in parallel

May require a larger heap size

☒ Rebuild module on dependency change

Build Process

Shared heap size:

Mbytes

Shared VM options:

User-local heap size:

Mbytes

Overrides Shared size

User-local VM options:

Overrides Shared options

OK

Cancel

App

检查这部分的配置

元宝:

好的，根据您提供的IDEA编译器配置界面截图，我们来详细检查并分析每一项配置。这个配置界面主要控制着 IntelliJ IDEA如何编译您的项目。

以下是针对图中各项配置的分析和建议：

配置分析与建议

1. Resource patterns (资源模式)

- **当前值**：`?.form;!?.class;!?.groovy;!?.scala;!?.flex;!?.kt;!?.clj;!?.aj`
- **分析**：这是一个过滤器，告诉IDEA哪些文件在编译时应被视为“资源文件”而被复制到输出目录（如 `target/classes`），而不是被编译器处理。
 - `?.form`：匹配所有 `.form` 文件（如Java Swing的GUI设计文件）。
 - `!?.class`：排除所有 `.class` 文件（已编译的Java字节码）。
 - 后续的 `!?` 开头的模式排除了其他语言（Groovy, Scala, Kotlin等）的源文件。
- **建议**：保持默认。这个配置非常完善，确保了只有真正的资源文件（如 `.properties`，`.xml`，`.json`）会被复制。如果您项目中有自定义的非标准资源文件扩展名，可以在这里添加。

2. Clear output directory on rebuild (在重建时清空输出目录)

- **分析**：这是一个复选框选项。如果勾选，每次执行 **Rebuild Project** 时，IDEA会先彻底删除 `out` 或 `target` 目录，然后重新编译。这可以确保得到一个绝对干净的构建，避免陈旧的编译文件引发问题。
- **建议**：建议勾选。这是一个很好的实践，可以避免很多因缓存或残留文件导致的诡异问题。请注意旁边的警告：它会清除生成源文件目录的全部内容。

3. Add runtime assertions for notnull-annotated methods and parameters (为NotNull注解添加运行时断言)

- **分析**：如果勾选，IDEA会在编译时为使用了 `@NotNull` 注解的方法和参数生成运行时断言检查。如果违反（即传入了 `null` 值），会抛出 `IllegalArgumentException`。
- **建议**：根据情况选择。在开发调试阶段建议勾选，有助于快速定位空指针问题。在生产环境构建时，可以取消勾选以获得轻微的性能提升。

4. Automatically show first error in editor (自动在编辑器中显示第一个错误)

- **分析**：编译失败后，是否自动在编辑器中定位并跳转到第一个编译错误处。
- **建议**：建议勾选。非常方便，能让你快速定位和修复错误。

5. Display notification on build completion (构建完成时显示通知)

- **分析**：构建过程结束后，是否在IDE右下角弹出提示框。
- **建议**：根据个人喜好。如果构建耗时较长，勾选后可以让你在等待时去做别的事，完成后会有通知。如果构建很快，可能会觉得提示框烦人。

6. Build project automatically (自动构建项目)

- **分析**：著名的“省电模式”或“自动编译”。勾选后，IDEA会在检测到文件更改时自动执行编译。
- **建议**：通常不建议勾选。虽然方便，但会持续消耗CPU和系统资源进行后台编译，可能影响性能。手动触发编译（`Ctrl+F9`）通常是更高效的选择。旁边的备注也提示“仅在非运行/调试时工作”。

7. Compile independent modules in parallel (并行编译独立模块)

- **分析**：对于多模块项目（如您的 `mall-swarm` ），IDEA会尝试并行编译那些没有依赖关系的模块，以加快编译速度。
- **建议**：**强烈建议勾选**。这能显著提升大型项目的编译效率。请注意旁边的备注“可能需要更大的堆大小”，这与您之前遇到的**内存不足问题**直接相关。

8. Rebuild module on dependency change (依赖变更时重建模块)

- **分析**：当一个模块所依赖的其他模块被重新编译后，是否自动重建该模块。
- **建议**：**建议勾选**。这能确保模块间的依赖关系始终保持最新，避免运行了过时的代码。

Build Process (构建进程配置 - 关键！)

这部分配置与您之前遇到的**内存不足错误**直接相关，请重点关注。

9. Shared heap size (共享堆大小)

- **当前值**：`4096` Mbytes (即 4GB)
- **分析**：这是分配给IDEA**编译进程**的最大堆内存。并行编译、处理大型项目（如 `mall-swarm` ）需要大量内存。
- **建议**：**4GB的设置是合理且充足的**。如果您再次遇到 `internal java compiler error` 或 `insufficient memory` 错误，可以尝试**进一步增大此值**（例如设置为 `5120` 或 `6144` ），前提是您的物理内存足够（建议机器至少有16GB物理内存）。

10. Shared VM options (共享VM选项)

- **当前值**：`空`
- **分析**：可以在此处传入额外的JVM参数给编译进程。
- **建议**：**可以保持为空**。如果您有特殊需求，可以在这里添加参数，例如指定垃圾回收器：`-XX:+UseG1GC`。

11. User-local heap size / VM options (用户本地堆大小/VM选项)

- **分析**：这些设置允许为当前项目覆盖全局的“Shared”设置。
- **建议**：**通常不需要设置**。除非某个特定项目有非常特殊的内存或配置需求。

总结与操作建议

您的编译器配置**整体上是合理和优化的**，特别是：

1. 开启了**并行编译**以提升速度。
2. 设置了**4GB的大内存**以支持大型项目的编译。

如果之后再次出现编译内存不足的问题，您的排查和调整顺序应该是：

1. **首先**：确认您使用的是 **64位JDK**（您已确认是Java 21 64-bit）。
2. **然后**：尝试在本次截图中的 `Shared heap size` 栏位，将值从 `4096` 增加到 `5120`（5GB）或 `6144`（6GB）。

3. **最后**：如果问题依旧，可以尝试在 **Shared VM options** 中增加一些优化参数，例如：

```
-XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=F:\heapdumps
```

（这会在内存溢出时生成堆转储文件，便于后续分析）。

配置完成后，点击 **OK** 保存即可。这些设置会立即生效。

（注：文档部分内容可能由 AI 生成）